



Abschlussprojekt

Projektunterlagen für Lernende

Fachinformatiker Anwendungsentwicklung · LF 8

Schuljahr 2025/2026

Inhaltsverzeichnis

Teamübersicht

Grundbegriffe für das Abschlussprojekt

 Nachschlagewerk

Projektauftrag

 Pflicht

Meilensteine und Termine

 Pflicht

Bündelungsfächer AE

 Pflicht

Abgabe und Repository

 Nachschlagewerk

Präsentation und Abschluss

 Nachschlagewerk

Vertretungsauftrag 10.06.

 Nachschlagewerk

Reflexion – Projektabschluss LF8

 Nachschlagewerk

Vertretungsauftrag 24.06.

 Nachschlagewerk

Teamübersicht

Diese Datei hält die Teamzusammensetzung fest. Die Rollenverteilung wird im Projektauftrag und in der Lehrkraftplanung vorgeschlagen und kann hier bei Bedarf ergänzt werden.

Team 1

- Kseniia U.
- Fynn S.
- Finn O.

Team 2

- Danilo W.
- Steven B.
- Oliver J.

Team 3

- Moritz T.
- Jeremy P.
- Marc P.

Team 4

- Daniel G.
- Jonas S.
- Finn J.

Team 5

- Jakob S.
- Sam D.
- Lennard S.

Team 6

- Nils B.
- Samuel W.

Hinweis für das 2er-Team

Das 2er-Team arbeitet mit reduziertem Pflichtumfang. Verbindlich sind zwei heterogene Quellen, ein gemeinsames Datenmodell, eine funktionsfähige API und eine regelbasierte Zuordnung.

Grundbegriffe für das Abschlussprojekt

Diese Datei erklärt die wichtigsten Begriffe, die im Projekt immer wieder auftauchen. Lies sie einmal durch, bevor du in die anderen Dokumente einsteigst.

User Story

Eine User Story beschreibt eine fachliche Anforderung aus der Sicht einer Person, die das System nutzt. Das Format ist immer:

Als [Rolle] möchte ich [Ziel], damit [Nutzen].

Beispiele

Als Lernende möchte ich meine Note in einem Portal einsehen können, damit ich nicht auf den Postweg warten muss.

Als Lehrkraft möchte ich sehen, welchen Fächern eine User Story zugeordnet wurde, damit ich die Bewertung transparent nachvollziehen kann.

User Story oder Ticket?

Im nächsten Schuljahr arbeiten **Anwendungsentwickler** eher mit **User Stories** (Was soll das System können?), während Systemintegratoren eher technische **Tickets** bearbeiten (Wie genau soll etwas umgesetzt werden?). In diesem Projekt sagen wir deshalb meistens **User Story**, meinen aber immer beides.

Datenquellen

Im Projekt meint "Quelle" eine **Datei in einem bestimmten Format**, aus der eure Anwendung Daten liest. Ihr simuliert damit, dass in der Realität verschiedene Tools ihre Daten unterschiedlich exportieren.

Konkrete Beispiele

- **Microsoft Planner** kann Aufgaben als CSV exportieren
- **GitHub Issues** lassen sich als JSON exportieren
- **Ältere oder XML-basierte Systeme** liefern XML-Dateien

Für euer Projekt benutzt ihr **selbst erstellte Beispieldateien**, die aber so aufgebaut sind, als kämen sie aus echten Tools.

Welche Formate benutzt ihr?

Format	Dateiendung	Kurzbeschreibung
CSV	.csv	tabellarisch, Felder durch Komma oder Semikolon getrennt
JSON	.json	strukturiert, weit verbreitet in Web-APIs
XML	.xml	strukturiert mit Tags, älterer Standard

Mindestens **zwei** dieser Formate müssen unterschiedliche Feldnamen oder unterschiedliche Strukturen haben. Genau das macht sie "heterogen".

Datenmodell

Ein Datenmodell legt fest, **welche Felder eine User Story in eurem System hat** – unabhängig davon, aus welcher Quelle sie stammt.

Warum braucht man das?

Quelle A (CSV-Export aus Planner) hat ein Feld `Aufgabe`. Quelle B (JSON-Export aus GitHub) hat ein Feld `title`. Beide meinen dasselbe. Im gemeinsamen Modell heißt das Feld bei euch z. B. `titel`.

Beispiel für ein einfaches gemeinsames Modell

Feld	Typ	Pflicht?	Erklärung
<code>id</code>	Text	ja	eindeutige Kennung
<code>titel</code>	Text	ja	Kurztitel der User Story
<code>beschreibung</code>	Text	nein	ausführlichere Erklärung
<code>quelle</code>	Text	ja	woher kam die Datei? <code>csv</code> , <code>json</code> oder <code>xml</code>
<code>rolle</code>	Text	nein	Fachrichtung oder Rolle (z. B. <code>AE</code>)
<code>priorität</code>	Text	nein	z. B. <code>hoch</code> , <code>mittel</code> , <code>niedrig</code>
<code>tags</code>	Liste	nein	Stichworte für die Zuordnung

Das Modell entscheidet ihr selbst – es muss aber für alle Quellen und für eure Fachzuordnung sinnvoll sein.

Mapping

Mapping bedeutet: "Welches Feld in Quelle A entspricht welchem Feld in unserem Modell?"

Beispiel

Feld in CSV-Quelle	Feld in JSON-Quelle	Feld in eurem Modell
Aufgabe	title	titel
Beschreibung	body	beschreibung
Priorität	priority	priorität
– (fehlt)	labels	tags

Wenn ein Feld in einer Quelle fehlt, schreibt ihr `None` oder einen Standardwert rein.

Das Mapping schreibt ihr zuerst als **Tabelle auf Papier oder in Markdown**, bevor ihr anfangt zu coden. Das ist die wichtigste Designarbeit vor der Implementierung.

API und Endpunkte

Eine API (Application Programming Interface) ist eine Schnittstelle, über die andere Programme oder Browser euren Dienst nutzen können. Ihr baut eine sogenannte **REST-API**.

Analogy

Stellt euch eine API wie die Karte eines Restaurants vor: - Jeder Eintrag auf der Karte ist ein **Endpunkt**. - Der Gast (ein Browser oder ein anderes Programm) schickt eine **Anfrage** (Bestellung). - Das Restaurant (eure Anwendung) schickt eine **Antwort** (Essen, oder eine Fehlermeldung).

Einfache Beispiele für Endpunkte

Methode	Pfad	Was passiert?
GET	/userstories	gibt alle gespeicherten User Stories zurück
GET	/userstories/42	gibt die User Story mit der ID 42 zurück
POST	/import/csv	ladet eine CSV-Datei hoch und liest sie ein
POST	/classify	schickt eine User Story rein, bekommt Fachzuordnung zurück

Wie testet ihr das?

FastAPI erzeugt automatisch eine Seite unter `http://localhost:8000/docs`, auf der ihr jeden Endpunkt direkt im Browser ausprobieren könnt. Das nennt man **Swagger UI**.

API-Skizze

Eine API-Skizze ist keine fertige Dokumentation, sondern eine **frühe Planung** auf Papier oder in Markdown. Sie beantwortet:

- Welche Endpunkte brauchen wir?
- Was schicken wir rein?
- Was kommt raus?

Beispiel

```
POST /import/csv
```

```
Eingabe: CSV-Datei als Upload
```

```
Ausgabe: Anzahl importierter User Stories
```

```
GET /userstories
```

```
Eingabe: keine
```

```
Ausgabe: Liste aller User Stories im gemeinsamen Format
```

```
POST /classify
```

```
Eingabe: eine User Story als JSON
```

```
Ausgabe: SDM-Anteil, EvP-Anteil, GiD-Anteil + kurze Begründung
```

Aufgabenboard

Ein Aufgabenboard ist eine Übersicht, wer gerade woran arbeitet und was noch offen ist. Es verhindert, dass zwei Personen die gleiche Aufgabe machen oder nichts mehr zu tun haben.

Option 1: GitHub Issues (empfohlen für dieses Projekt)

Ihr habt bereits GitHub-Accounts. Das Repository, das ihr anlegt, kann direkt GitHub Issues nutzen:

1. Repository anlegen
2. Tab "Issues" öffnen
3. Für jede Aufgabe ein Issue anlegen: kurzer Titel, kurze Beschreibung, wer zuständig ist
4. Issue schließen, wenn die Aufgabe fertig ist

Option 2: Microsoft Planner (gut geeignet für das Folgeprojekt)

Ihr habt alle M365-Accounts. Microsoft Planner ist Teil davon und funktioniert ähnlich wie ein digitales Aufgaben-Board (Kanban-Prinzip):

1. Planner über Teams oder office.com öffnen
2. Neuen Plan anlegen (Name = euer Projektname)
3. Buckets anlegen (z. B. "Offen", "In Arbeit", "Fertig")
4. Für jede Aufgabe eine Karte erstellen: Titel, zuständige Person, Fälligkeitsdatum
5. Karte in den richtigen Bucket verschieben, wenn sich der Status ändert

Hinweis für das kommende Schuljahr: Im großen Projektübergreifenden Projekt werden wir Microsoft Planner einsetzen. Wer das schon jetzt ausprobiert, ist gut vorbereitet.

Option 3: einfache Aufgabenliste im Teamlog

Falls ein digitales Board zu aufwendig wirkt, reicht auch eine gepflegte Tabelle in `doku/teamlog.md`. Wichtig ist nur, dass ihr zu jedem Termin seht: Was ist offen, wer macht was?

Dokumentation – Was gehört wohin?

Ihr schreibt drei Arten von Dokumentation. Jede hat eine andere Zielgruppe:

README.md – für jeden

Die README ist die erste Datei, die jemand liest, der euer Projekt öffnet. Sie beantwortet:

- Was macht das Projekt?
- Wie startet man es?
- Was wurde gebaut?

Zielgruppe: Die Lehrkraft, ein fremdes Team, zukünftige Nutzer.

doku/api.md – für Entwickler und Tester

Diese Datei erklärt die API-Endpunkte:

- Welche Routen gibt es?
- Welche Daten müssen rein?
- Was kommt raus?
- Welche Fehlercodes gibt es?

Zielgruppe: Jemand, der eure API benutzen will, ohne den Code zu lesen.

doku/teamdoku.md – für das Projekt selbst

Diese Datei dokumentiert eure Arbeit und Entscheidungen:

- Warum habt ihr das Datenmodell so gewählt?
- Welche Probleme gab es?
- Wer hat was gemacht?

Zielgruppe: Die Lehrkraft bei der Bewertung und ihr selbst beim Nachschlagen.

Glossar kurz

Begriff	Bedeutung
Heterogen	unterschiedlich aufgebaut, verschiedene Formate oder Feldnamen
Normalisieren	auf eine gemeinsame, einheitliche Struktur bringen
Endpoint / Endpunkt	eine aufrufbare Adresse eurer API
Mapping	Zuordnungstabelle: Quellfeld → Zielfeld
Regelbasiert	keine KI, sondern selbst definierte Regeln und Bedingungen
Swagger UI	automatische Testoberfläche, die FastAPI generiert
Repository / Repo	euer gemeinsamer Code-Ordner auf GitHub
Commit	ein gespeicherter Änderungsschritt im Repository

Projektauftrag

🎯 **Ziel für Woche 1 (27.05.):** Team zusammenstellen, zwei Quellformate benennen, erste Feldliste skizzieren. Alles andere kommt danach – die restlichen Kapitel sind Nachschlagewerk.

Neu im Projektarbeiten? Lies zuerst 00 - Grundbegriffe. Dort werden Begriffe wie "User Story", "Datenquelle", "Mapping" und "API-Endpunkt" erklärt.

Ausgangssituation

Im nächsten Schuljahr arbeitet ihr in größeren, fächerübergreifenden Projektteams. Damit ein solches Projekt später transparent bewertet werden kann, müssen **User Stories** verschiedenen Bündelungsfächern zugeordnet werden – also: Welches Thema steckt in dieser Aufgabe, und für welches Fach ist das relevant?

In diesem Abschlussprojekt entwickelt ihr einen ersten, kleineren Prototyp für genau diese Aufgabe.

Was sind die Datenquellen?

Als "Datenquellen" dienen Dateien, die ihr selbst erstellt. Sie simulieren Exporte aus echten Werkzeugen:

- eine **CSV-Datei** (z. B. so wie sie Microsoft Planner exportieren würde)
- eine **JSON-Datei** (z. B. so wie GitHub Issues exportiert werden)
- optional: eine **XML-Datei** (z. B. ein Export aus einem älteren Projekttool)

Die Felder in diesen Dateien heißen absichtlich unterschiedlich – das erzeugt die "Heterogenität", die ihr lösen sollt.

Ziel

Entwickelt als Team einen Dienst, der:

1. User Stories aus heterogenen Quellen (CSV, JSON, optional XML) einliest,
2. die Daten in ein gemeinsames, einheitliches Modell überführt,
3. die normalisierten Daten per API bereitstellt,
4. eine regelbasierte Empfehlung für die Zuordnung zu SDM, EvP und GiD erzeugt.

Rollenlogik im Team

Es gibt keine feste Trennung nach Frontend, Backend und n8n. Stattdessen arbeitet ihr als Produktteam mit drei Schwerpunkten:

Rolle A - Datenimport und Mapping

- analysiert die Formate (CSV, JSON, XML)
- plant das gemeinsame Datenmodell
- überführt die Quelldaten in die Zielstruktur

Rolle B - Fachlogik und Regelwerk

- entwickelt die Regeln für die Fachzuordnung
- testet Beispiel-User-Stories gegen die Regeln
- dokumentiert Begründungen und Grenzfälle

Rolle C - API, Validierung, Tests und Doku

- erstellt die Endpunkte
- prüft Eingaben und Ausgaben
- sorgt für Testprotokoll und Startdokumentation

Wichtig: Jede Person arbeitet schwerpunktmäßig in einem Bereich, aber jedes Teammitglied muss den Gesamtaufbau erklären können.

Muss-Anforderungen

Für alle Teams

- eigenes Team-Repository
- mindestens zwei heterogene Quellen (unterschiedliche Formate und/oder unterschiedliche Feldnamen)
- gemeinsames Datenmodell mit Mapping-Tabelle
- API mit mindestens zwei sinnvollen Endpunkten
- regelbasierte Zuordnung zu SDM, EvP und GiD
- kurze technische Doku (README + api.md)
- Teamlog mit sichtbaren Einzelbeiträgen

Zusätzlich für Dreierteams

- dritte Quelle nach Möglichkeit integrieren (in der Regel XML)

Für das Zweierteam

- zwei Quellen genügen als verbindlicher Kern
- Fokus auf Stabilität statt Umfang

Soll-Anforderungen

- Fehlerbehandlung für fehlerhafte Eingaben
- einfache Tests oder Testprotokolle

- klare Regeln mit Positiv- und Negativbeispielen für die Fachzuordnung
- saubere Projektstruktur

Kann-Anforderungen

Erst nach erfolgreicher Kernabnahme:

- Frontend
- n8n-Workflow
- Exportfunktion
- Hosting
- Docker
- weitere API-Endpunkte

Technischer Rahmen

Empfohlen:

- Python
- FastAPI
- Pydantic
- Standardbibliothek für CSV, JSON und XML

Andere sinnvolle Lösungen sind möglich, solange sie für euch beherrschbar bleiben.

Definition of Done für die Kernabnahme

Euer Kern ist fertig, wenn ihr live zeigen könnt:

1. mindestens zwei Quellen werden eingelesen,
2. die Daten sind in ein gemeinsames Modell übertragen,
3. die API liefert die Daten,
4. die Fachzuordnung arbeitet nachvollziehbar,
5. euer Team kann Aufbau und Entscheidungen erklären.

Meilensteine und Termine

Neu im Projektarbeiten? Lies zuerst 00 - Grundbegriffe, bevor du hier weitermachst. Viele Begriffe wie "Datenmodell", "Mapping" oder "Endpunkte" werden dort erklärt.

Übersicht

Termin	Ziel	Was am Ende sichtbar sein muss
27.05.	Projektstart	Team, Quellen, Rollen, erste Modellidee
03.06.	Konzept steht	Datenmodell, Mapping-Tabelle, Regelideen, API-Skizze
10.06.	Vertretungssprint	Mapping fertig, Regelwerk V1, API-Plan, Aufgabenplan
17.06.	Technikreview	erster lauffähiger Kern
24.06.	Kernabnahme	benotbarer Kernstand
08.07.	Politur (45 min)	Präsentation und Demo vorbereitet
15.07.	Abschluss	Demo, Präsentation, Reflexion
## 27.05. - Projektstart		

Was ihr heute macht

Schritt 1: Team und Name

Legt einen Projektnamen fest. Macht das nicht zu ernst – ein kurzer, einprägsamer Name reicht.

Schritt 2: Quellen auswählen

Ihr braucht mindestens zwei Dateien in **unterschiedlichen Formaten** (z. B. eine CSV-Datei und eine JSON-Datei), die User Stories enthalten. Diese Dateien erstellt ihr selbst – sie simulieren Exporte aus realen Tools.

Konkrete Idee: eine CSV-Datei, die wie ein Planner-Export aussieht, und eine JSON-Datei, die wie ein GitHub-Issues-Export aussieht.

Falls ihr nicht wisst, wie ein solcher Export aussieht, versucht welche zu erzeugen (Github Export am besten über CLI per `gh issue list --repo *benutze/reponame* --json number,title,state,labels,assignees,createdAt > *dateiname.json*`).

Schritt 3: Rollen verteilen

Wer übernimmt welchen Schwerpunkt? Schreibt das auf:

- Rolle A (Datenimport und Mapping): ____
- Rolle B (Fachlogik und Regelwerk): ____

- Rolle C (API, Tests, Doku): ____

Hinweis für das 2er-Team: Ihr habt eine Person weniger als die anderen Teams. Das ist kein Nachteil – euer Projekt darf entsprechend kompakter sein. Überlegt, welche zwei Rollen ihr als Schwerpunkte wählt und wo ihr zusammenarbeitet. Empfehlung: Person 1 übernimmt Rolle A + halbe Rolle B, Person 2 übernimmt Rolle C + halbe Rolle B. Sprecht das in eurem ersten Review kurz an.

Schritt 4: Gemeinsames Datenmodell skizzieren

Überlegt: Wenn ihr eine User Story aus eurer CSV-Quelle und eine aus eurer JSON-Quelle in denselben "Topf" werft – welche Felder soll dieses einheitliche Datenformat haben?

Schreibt eine erste Tabelle, zum Beispiel:

Feld	Pflicht?	Beispiel
id	ja	"US-01"
titel	ja	"Login implementieren"
beschreibung	nein	"Als Nutzer möchte ich..."
quelle	ja	"csv" oder "json"
priorität	nein	"hoch"

Das ist noch kein Code – nur eine Skizze auf Papier oder in Markdown.

Schritt 5: Aufgabenboard anlegen

Öffnet euer GitHub-Repository und legt unter dem Reiter "Issues" erste Aufgabenkarten an. Jede Aufgabe = ein Issue. Oder: Füllt die Tabelle im Teamlog. Wichtig ist, dass ihr seht, wer was bis zum nächsten Termin erledigt.

Was am Ende der Stunde vorliegt

- Teamname
- zwei benannte Quellen mit konkreten Formaten
- Rolleneinteilung
- erste Feldliste für das gemeinsame Modell
- Repository angelegt und Teamlog gestartet

03.06. - Konzept steht

An diesem Termin gibt es kurze Gespräche mit der Lehrkraft pro Team (ca. 10 Minuten). Die anderen Teams arbeiten weiter.

***Format des Gesprächs:** Jede Person präsentiert kurz und knackig ihr bisheriges Ergebnis – ca. 2 Minuten pro Person, danach einige Fachfragen von der Lehrkraft. Kein langes Herumsuchen in Dateien! Zeigt nur das, was ihr wirklich vorweisen könnt – egal ob auf dem Bildschirm, auf Papier oder mündlich. Das Präsentationsmittel ist egal, aber die Zeit ist knapp. Bereitet euch so vor, dass ihr sofort auf den Punkt kommt.*

Was ihr bis dahin fertig habt

Beispieldateien liegen vor

Ihr habt mindestens zwei echte Beispieldateien erstellt, z. B.:

- `data/userstories_planner.csv` mit 5 bis 10 User Stories
- `data/userstories_github.json` mit 5 bis 10 User Stories

Die Felder sind bewusst unterschiedlich benannt, damit ihr das Mapping üben müsst.

Mapping-Tabelle ist fertig

Ihr habt eine Tabelle gebaut (in Markdown oder auf Papier), die zeigt: Welches Feld in Quelle A entspricht welchem Feld in eurem gemeinsamen Modell? → Siehe 00 - Grundbegriffe für ein Beispiel.

API-Skizze ist fertig

Eine einfache Liste mit 3 bis 4 geplanten Endpunkten. Was soll die API können? Was geht rein, was kommt raus? Kein Code nötig, nur Text. → Erklärt in 00 - Grundbegriffe.

Erste Regeln für SDM, EvP und GiD sind formuliert

Mindestens 3 Regeln pro Fach: Welche Begriffe im Titel oder in der Beschreibung einer User Story deuten auf welches Fach hin? → Details in 03 - Bündelungsfächer AE.

Was ihr beim Review zeigen können müsst

- Beispieldateien (öffnen und zeigen)
- Mapping-Tabelle (erklären, welche Felder zusammenpassen)
- 3 bis 5 Regelideen (woran erkennt ihr SDM, EvP oder GiD?)
- geplante Endpunkte (welche Adressen soll eure API haben?)

10.06. - Vertretungssprint

Kein Unterricht im klassischen Sinne. Ihr arbeitet selbstständig. Der genaue Auftrag steht in 06 - Vertretungsauftrag 10.06..

17.06. - Technikreview

An diesem Termin zeigt ihr zum ersten Mal echten, laufenden Code.

Erwarteter Stand

- Import aus mindestens einer, besser zwei Quellen läuft
- Daten werden in das gemeinsame Modell übertragen
- erste API-Antwort funktioniert (auch wenn nur ein Endpunkt)
- erste Zuordnung für Beispiel-User-Stories läuft
- offene Probleme sind benannt

Ziel des Termins

Nach dem Gespräch soll klar sein: Was fehlt noch bis zum 24.06.?

24.06. - Kernabnahme

Dies ist der entscheidende Termin. **Die benotbare Basis muss heute stehen.**

Am 24.06. findet Vertretung statt. Es gilt verbindlich der schriftliche Auftrag aus 08 - Vertretungsauftrag 24.06..

Wenn zusätzlich die 3./4. Stunde vertreten wird, arbeitet ihr in Abstimmung mit der vertretenden Lehrkraft weiter am Projekt.

Für weitere Vertretungsstunden gilt: Arbeitet am Projekt weiter, sofern keine abweichenden Aufgaben der vertretenden Lehrkraft vorliegen.

Bewertet wird

- stabile Kernfunktion (Import, Modell, API, Zuordnung)
- Erklärbarkeit: Kann jede Person ihren Teil erklären?
- sichtbarer Individualbeitrag
- Doku: Kann man das Projekt ohne Hilfe starten?

Noch nicht entscheidend

- fertiges Frontend
- n8n-Anbindung
- besonders viele Endpunkte
- Hosting oder Docker

08.07. - 45 Minuten Politur

Nutzt die Zeit für:

- Generalprobe der Demo
- Doku fertigstellen
- letzte Fehler beheben
- Präsentation durchgehen

15.07. - Abschluss

Abgeben und zeigen

- finaler Stand im Repository
- Präsentation
- Live-Demo
- individuelle Reflexion ausgefüllt

Bündelungsfächer AE

Diese Datei gibt euch für dieses Projekt eine transparente Arbeitsdefinition der drei Bündelungsfächer. Sie ersetzt keine vollständige Bildungsplananalyse, schafft aber eine nachvollziehbare gemeinsame Grundlage für eure regelbasierte Zuordnung.

SDM - Softwaretechnologie und Datenmanagement

Im Projekt meint das vor allem

- Datenstrukturen und Datenmodelle
- Parsing, Mapping und Vereinheitlichung
- Umgang mit CSV, JSON und XML
- Qualität von Daten
- technische Struktur der Anwendung

Typische Hinweise in Tickets

- Datenmodell
- Feldzuordnung
- Import
- JSON
- CSV
- XML
- Validierung
- Datensatz
- Struktur

Beispiel

"Als System sollen CSV-Daten in ein gemeinsames Ticketformat überführt werden."

Das spricht stark für SDM.

EvP - Entwicklung vernetzter Prozesse

Im Projekt meint das vor allem

- Schnittstellen
- API-Endpunkte
- Datenaustausch zwischen Systemen
- Automatisierung von Übergängen
- Prozessfluss vom Eingang bis zur Ausgabe

Typische Hinweise in Tickets

- API
- REST
- Endpunkt
- Anfrage
- Antwort
- Webhook
- Workflow
- Integration
- Weiterverarbeitung

Beispiel

"Als externer Dienst möchte ich die vereinheitlichten Tickets über eine API abrufen."

Das spricht stark für EvP.

GiD - Gestaltung von IT-Dienstleistungen

Im Projekt meint das vor allem

- Kundenperspektive
- Nutzbarkeit und Verständlichkeit
- Doku, Übergabe und Handhabung
- klare Anforderungen
- sinnvolle Darstellung der Ergebnisse

Typische Hinweise in Tickets

- Nutzer
- Kunde
- Dokumentation
- Bedienung
- Übergabe
- Beschreibung
- Verständlichkeit
- Darstellung

Beispiel

"Als Kunde möchte ich eine kurze Anleitung erhalten, damit ich die API ohne Rückfragen nutzen kann."

Das spricht stark für GiD.

Arbeitsregeln für eure Zuordnung

1. Eine Regel muss begründbar sein.
2. Ein Ticket darf zu mehreren Fächern passen.
3. Gebt bei Mischfällen eine gewichtete Empfehlung ab.
4. Nutzt Positiv- und Negativbeispiele.
5. Dokumentiert unklare Fälle offen.

Einfache Gewichtungsidee

- starker Hinweis = 2 Punkte
- schwacher Hinweis = 1 Punkt
- Gegenhinweis = 0 oder Abzug nach eigener Logik

Beispiel für eine Ausgabe

Fach	Punkte	Begründung
SDM	3	CSV-Import, Feldmapping, Datenmodell
EvP	2	API-Abruf und Weitergabe
GiD	1	kurze Nutzungsanleitung für den Kunden

Wichtig

Eure Regeln müssen nicht "perfekt intelligent" sein. Sie müssen für euer Projekt nachvollziehbar, konsistent und erklärbar sein.

Abgabe und Repository

Ein Repository pro Team

Jedes Team arbeitet in einem eigenen Repository. Ziel ist nicht ein komplizierter Git-Workflow, sondern eine saubere Zusammenarbeit.

Minimale Ordnerstruktur

```
team-x/  
├─ README.md  
├─ src/  
├─ data/  
├─ examples/  
├─ doku/  
│   ├─ teamdoku.md  
│   ├─ api.md  
│   └─ tests.md  
└─ teamlog.md  
└─ requirements.txt
```

Pflichtdateien

Kurze Erklärung: Wofür ist welche Datei?

Ihr schreibt drei Arten von Dokumentation – jede für eine andere Zielgruppe:

Datei	Zielgruppe	Frage, die sie beantwortet
README.md	alle, die euer Repo öffnen	Was macht das Projekt, und wie starte ich es?
doku/api.md	Entwickler und Tester	Wie benutze ich die API? Welche Endpunkte gibt es?
doku/teamdoku.md	Lehrkraft, ihr selbst	Warum habt ihr so entschieden? Wer hat was gemacht?

README.md

Zielgruppe: **alle**, die euer Repository öffnen.

- Projektname und kurze Beschreibung (2–3 Sätze)
- Teammitglieder und Rollen
- Systemvoraussetzungen (Python-Version, Bibliotheken)

- Startanleitung: Was muss ich tun, damit die API läuft?
- bekannter Stand: Was funktioniert, was fehlt noch?

doku/teamdoku.md

Zielgruppe: **Lehrkraft und euer Team** – erklärt Entscheidungen und Zusammenarbeit.

- verwendete Quellen und deren Formate
- Datenmodell mit Begründung (Warum diese Felder?)
- Mapping-Tabelle (Quelle → Modell)
- Rollen und Aufgabenverteilung
- Herausforderungen und wie ihr sie gelöst habt

doku/api.md

Zielgruppe: **jemand, der eure API benutzen will**, ohne den Code zu lesen.

- alle Endpunkte mit HTTP-Methode und Pfad
- Was wird als Eingabe erwartet?
- Was gibt die API zurück?
- Fehlerfälle und Fehlermeldungen
- Beispielanfragen und Beispiellantworten

doku/tests.md

- Testfälle mit Beschreibung
- erwartete Ergebnisse
- tatsächliche Ergebnisse
- Status: bestanden / fehlgeschlagen

doku/teamlog.md

- was wurde wann von wem bearbeitet
- offene Punkte
- nächster Schritt

Einfache Git-Regeln

1. `main` soll möglichst lauffähig bleiben.
2. Macht kleine, sinnvolle Commits.
3. Vermeidet es, gleichzeitig dieselbe Datei zu bearbeiten.
4. Holt regelmäßig den aktuellen Stand.
5. Klare Commit-Nachrichten sind wichtiger als komplizierte Git-Technik.

Sinnvolle Commit-Beispiele

- `Import CSV vorbereitet`
- `Regelwerk für SDM und EvP ergänzt`
- `GET /userstories fertiggestellt`
- `README und API-Beispiele ergänzt`

Was bis 24.06. im Repo sichtbar sein muss

- Arbeitsstand des Kerns
- Rollenbeitrag jeder Person
- Doku
- mindestens zwei funktionierende Quellen
- API und Zuordnung

Zusatz bei Vertretung am 24.06. - schriftliche Kernstandsabgabe

Am 24.06. findet Vertretung statt. Daher gebt ihr an diesem Tag verbindlich eine strukturierte schriftliche Kernstandsabgabe ab.

Pflichtdatei

- `doku/abgabe_2026-06-24.md` (genaue Struktur siehe 08 - Vertretungsauftrag 24.06.)

Teams-Abgabe

- Zusätzlich wird die komplette Projektordnerstruktur als ZIP in der terminierten Teams-Aufgabe hochgeladen.
- Die ZIP muss zum angegebenen Commit-Hash passen.

Abgaberegeln

- Die Datei muss vollständig sein und sich nur auf den aktuellen Stand im Repository beziehen.
- Jede Aussage muss durch Pfade, Dateien, Tests oder Beispiele belegbar sein.
- Fehlende Punkte werden offen benannt statt beschönigt.

Hinweis für weitere Vertretungsstunden

- In zusätzlichen Vertretungsstunden arbeitet ihr am Projekt weiter, sofern keine abweichenden Aufgaben der vertretenden Lehrkraft vorliegen.

Was erst danach sinnvoll ist

- Frontend
- n8n
- Zusatzfeatures

Wenn der Kern nicht steht, helfen Extras nicht weiter.

Präsentation und Abschluss

Ziel der Abschlussphase

Ihr zeigt nicht nur ein Ergebnis, sondern auch, dass ihr den Weg vom heterogenen Eingang bis zur nutzbaren Bereitstellung fachlich verstanden habt.

Zeitplan für den 15.07.

Es präsentieren **6 Teams** in einer **90-Minuten-Stunde**. Jeder Teamwechsel am Präsentationsgerät kostet Zeit. Deshalb gilt: **Strikte Zeitvorgaben**.

Phase	Zeit
Präsentation	5 Minuten
Live-Demo	3 Minuten
Rückfragen der Lehrkraft	2 Minuten
Wechsel, Aufbau, Puffer	2 Minuten
Summe pro Team	12 Minuten
6 Teams gesamt	72 Minuten
Verbleibend für Abschluss, Feedback, Wrap-up	ca. 18 Minuten

Die Uhr läuft ab dem ersten Wort. Wer vorher aufbaut, spart Zeit.

Tipps für die 5-Minuten-Präsentation

5 Minuten klingen kurz – mit klarer Struktur reichen sie:

Folie	Inhalt	Zeit
1	Was haben wir gebaut? (ein Satz)	30 Sek
2	Welche Quellen, welches Datenmodell?	60 Sek
3	Wie funktioniert die Zuordnung zu SDM/EvP/GiD?	90 Sek
4	Wer hat was gemacht?	30 Sek
5	Was würde das Team beim nächsten Mal anders machen?	30 Sek

Dann direkt in die Demo übergehen – **keine separate Einleitung**.

Demo-Vorbereitung

Die Demo soll **ohne Erklärung funktionieren**. Bereitet konkret vor:

- API-Aufruf mit einem Beispiel-Request liegt bereit (z. B. in Swagger UI oder als fertiger curl-Befehl)
- Ergebnis der Fachzuordnung ist sichtbar
- keine Fenster müssen erst gesucht oder geöffnet werden

Generalprobe am 08.07. ist Pflicht.

Was in der Präsentation gut wirkt

- max. 5 Folien, große Schrift
- eine Tabelle für das Datenmodell
- ein konkretes Beispiel für die Zuordnung (Eingabe → Ergebnis)
- die Demo als Höhepunkt, nicht als Anhang

Was ihr vermeiden solltet

- mehr als 5 Folien
- Code-Listing auf Folien (zeigt ihn live oder lasst es weg)
- Demo, die nicht läuft oder die ihr erst im letzten Moment aufbaut
- Extras erklären, die nicht zum Kern gehören

Endabgabe bis 15.07.

- finaler Stand im Repository (gepusht vor der Stunde)
- vollständige Pflichtdoku (README, api.md, teamdoku.md)
- Präsentation (5 Folien)
- Reflexion ausgefüllt (Forms-Link erhaltet ihr von der Lehrkraft)

Leitfrage für euren Abschluss

Könnte ein fremdes Team euer Produkt verstehen und die API ohne eure Hilfe benutzen?

Vertretungsauftrag 10.06.

Heute arbeitet ihr selbstständig an eurem Projekt weiter. Ziel ist kein Perfektionismus, sondern ein sauberer Zwischenschritt in Richtung Kernabnahme.

Teil A - Datenanalyse

Bearbeitet für eure Quellen:

- Welche Felder gibt es?
- Welche Felder sind gleich oder ähnlich?
- Welche Informationen fehlen in einzelnen Quellen?
- Welche Werteformate sind problematisch?

Ergebnis

Eine Mapping-Tabelle von Quelle A, Quelle B und optional Quelle C in euer gemeinsames Datenmodell.

Teil B - Regelwerk Version 1

Entwickelt erste Regeln für SDM, EvP und GiD.

Mindestumfang

- mindestens 8 Regeln
- mindestens 2 Beispiel-Tickets
- mindestens 1 Grenzfall

Ergebnis

Ein kurzer Regelkatalog mit Begründung.

Teil C - API-Plan

Plant:

- welche Endpunkte ihr braucht
- welche Daten ihr zurückgeben wollt
- welche Felder Pflicht sind
- welche Fehlerfälle bedacht werden müssen

Ergebnis

Eine API-Skizze mit Beispielantworten.

Teil D - Aufgabenplan bis 17.06.

Legt fest:

- wer macht was
- welche Dateien entstehen
- was bis zum Technikreview fertig sein muss

Ergebnis

Ein kurzer Plan für die nächsten Schritte.

Pflichtnachweise am Ende des Termins

- Mapping-Tabelle
- Regelwerk V1
- API-Skizze
- Aufgabenplan

Zusatzhinweis

Wenn ihr früher fertig seid, beginnt mit:

- erstem Import einer Quelle
- erster API-Rückgabe
- ersten Tests

Reflexion – Projektabschluss LF8

Die Reflexion wird als **Microsoft Forms-Umfrage** ausgefüllt. Den Link bekommt ihr von der Lehrkraft am 15.07.
Antworte ehrlich und kurz. Es gibt keine falschen Antworten – es geht um deinen persönlichen Lernprozess.

Übersicht der Fragen (zur Vorbereitung)

Du kannst dich hier schon einmal vorbereiten, bevor du den Link öffnest.

Abschnitt 1 – Mein Beitrag

1. Welche Rolle hattest du im Team? (*Auswahl*)
2. Datenimport und Mapping (Rolle A)
3. Fachlogik und Regelwerk (Rolle B)
4. API, Tests und Dokumentation (Rolle C)
5. Wechselnde Aufgaben / kein fester Schwerpunkt
6. Wie groß war dein eigener Anteil am Teamergebnis? (*Skala 1–5: sehr gering → sehr groß*)
7. Was war konkret dein wichtigster Beitrag? (*Kurztext, 1–2 Sätze*)

Abschnitt 2 – Zusammenarbeit

1. Wie gut hat die Teamarbeit insgesamt funktioniert? (*Skala 1–5*)
2. Wo gab es Schwierigkeiten? (*Mehrfachauswahl*)
3. Technische Probleme beim Import oder der API
4. Git / Konflikte im Repository
5. Aufgabenverteilung unklar
6. Kommunikation im Team
7. Zeitplanung
8. Keine nennenswerten Probleme

Abschnitt 3 – Fachliches Lernen

1. Was hast du in diesem Projekt wirklich verstanden? (*Mehrfachauswahl*)
2. Datenformate (CSV, JSON, XML) einlesen
3. Daten in ein einheitliches Modell überführen (Mapping)
4. API-Endpunkte mit FastAPI erstellen
5. Regelbasierte Fachzuordnung entwickeln
6. Git und kollaboratives Arbeiten
7. Technische Dokumentation schreiben
8. Wie gut könntest du euer Regelwerk für die Fachzuordnung jemandem erklären, der nicht im Projekt war?
(*Skala 1–5*)

Abschnitt 4 – Ausblick

1. Wie schätzt du deinen eigenen Lernfortschritt in diesem Projekt ein? (*Skala 1–5: kaum etwas gelernt → sehr viel gelernt*)
2. Was nimmst du für das nächste Projekt mit? (*Kurztext, 1 Satz – optional*)

Vertretungsauftrag 24.06.

Heute ersetzt die schriftliche Abgabe die mündliche Kernabnahme. Ziel ist ein belastbarer, prüfbarer Projektstand mit klaren Nachweisen.

Abgabeformat

Legt im Repository genau diese Datei an:

- `doku/abgabe_2026-06-24.md`

Verwendet die folgende Struktur und Reihenfolge verbindlich.

Abgabe in Teams

Zusätzlich zur Datei im Repository gebt ihr eure Arbeit in der terminierten Teams-Aufgabe ab:

- komplette Projektordnerstruktur als ZIP-Datei
- Dateiname: `teamname_2026-06-24.zip`
- die ZIP muss den gleichen Stand enthalten wie der angegebene Commit-Hash

Zusatz bei weiterer Vertretung

Wenn weitere Vertretungsstunden stattfinden, arbeitet ihr am Projekt weiter, sofern keine abweichenden Aufgaben der vertretenden Lehrkraft vorliegen.

Insbesondere bei einer Vertretung in der 3./4. Stunde am 24.06. arbeitet ihr in Abstimmung mit der vertretenden Lehrkraft weiter am Projekt.

1) Kopfdaten

- Teamname
- Mitglieder (Vorname Nachname)
- Datum
- Repository-URL
- letzter Commit-Hash vor Abgabe

2) Kernstatus in 6 Sätzen

Beantwortet knapp und präzise:

1. Welche Quellen werden verarbeitet?
2. Was funktioniert beim Import stabil?
3. Wie sieht euer gemeinsames Datenmodell aus?

- 4. Welche API-Endpunkte laufen bereits?
- 5. Wie funktioniert die Zuordnung zu SDM, EvP, GiD?
- 6. Was ist der wichtigste offene Punkt?

3) Nachweis Datenmodell und Mapping

3.1 Datenmodell

Tabelle mit mindestens 8 Feldern:

Feld	Datentyp	Pflicht (ja/nein)	Beispielwert

3.2 Mapping-Tabelle

Zielfeld	Quelle A Feld	Quelle B Feld	Regel/Transformation

4) Nachweis Regelwerk Fachzuordnung

Mindestens 10 Regeln insgesamt, verteilt auf SDM, EvP, GiD.

Regel-ID	Fach	Auslöser (Keyword/Muster)	Begründung	Beispiel-Story

Zusätzlich: 2 Grenzfälle mit kurzer Begründung, warum die Entscheidung schwierig ist.

5) Nachweis API und Tests

Dokumentiert mindestens 3 Endpunkte.

Methode + Pfad	Zweck	Beispielanfrage	Beispielantwort (gekürzt)	Status

Ergänzt mindestens 5 Testfälle:

Test-ID	Was wird geprüft?	Erwartet	Ergebnis	bestanden (ja/nein)

6) Individueller Beitrag je Person

Für jede Person eine Zeile:

Name	Hauptaufgabe	konkrete Dateien/Ordner	2 wichtige Commits	offener nächster Schritt

7) Fehler, Risiken, nächste Schritte

- 3 aktuell bekannte Fehler oder Risiken
- Priorität je Punkt (hoch/mittel/niedrig)
- konkreter Arbeitsplan bis 08.07.

8) Selbstcheck vor Abgabe

- ☐ Alle 7 Abschnitte vollständig
- ☐ Alle Tabellen ausgefüllt
- ☐ Beispiele sind echt und stammen aus unserem Stand
- ☐ Commit-Hash stimmt mit dem Abgabestand überein
- ☐ Offene Punkte sind ehrlich benannt

Mindeststandard

Eine unvollständige oder pauschale Abgabe gilt nicht als benotbarer Kernstand. Qualität entsteht durch Nachvollziehbarkeit, nicht durch lange Texte.